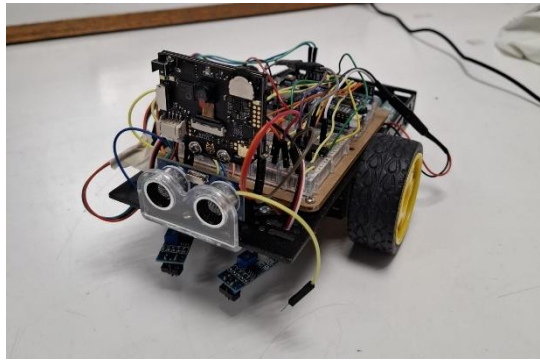


2E10 Final Report

Group <W01>



Fionan Nolan, Jack Dai, Maxim Chvets and Paula Brüdric

Contents

Part A – Self Assessment	3
Strengths:	3
Weaknesses:	3
Design Improvements:	4
Ethics:	4
Part B – Hardware Design	5
Block Diagram:	5
Circuit Diagram	6
Analysis of the board	6
Challenges	7
Part C – Software Design	8
Domain Model	8
System Model	9
Pseudo Code	10
Processing Pseudo Code	10
Arduino Pseudo Code	11
Part C Conclusion	15
Part D – Gold Challenge	16
Domain Model	16
System Model	17
Pseudo Code	18
Processing Pseudo Code	18
Arduino Pseudo Code	20
Part D Conclusion	24
Part E – Appendix	25
0. Additional Graphs and Images	25
3. Version History	27

Part A – Self Assessment

Strengths:

- Quick and easy communication between group members: The majority of the communication in the group was done online on a WhatsApp group chat. With quick responses consistently, asking and answering questions was easy and efficient.
- Allocation of tasks according with individual's strength: The workload was distributed, and tasks were allocated to members based on their specific skillset.
- Organizing consistent group meetings: Meetings outside of lab times were organised allowing for consistent progress to be made and for all group members to stay on top of their tasks.

Weaknesses:

- Debugging: When debugging an issue, a possible hypothesis would arise as to the origin of the bug. The group would fixate on this hypothesis rather than spending more time on looking at other possible origins of the bug. This used up more time than was necessary.
- Lack of alternative plans: In our opinion, the group did a good job at thinking of a plan, creating tasks that will carry out this plan in time for the demonstrations and distributing these tasks to group members. However, in case one of these tasks wasn't completed or the plan couldn't be followed, there was a lack of alternative plans, and was left to thinking on the spot. Luckily the group didn't encounter a situation where a pre-defined alternative plan was needed.
- Time management: On a few occasions, the group were rushing to finish off tasks in the final week before the deadline. The group had planned due dates in advance however, struggled to keep up with busy schedules often clashing. Although the group managed to complete all their given tasks, better time management would allow for less stress and additional time to review and reassess their work, coming up to the due date.

Design Improvements:

Design of the signs

This was against the rules of the gold challenge, so we didn't go through with it, however we had a plan to make the signs for the HuskyLens camera overhead the track rather than on the side of the track. This would make it easier for the buggy to predict when it will pass the sign rather than when the sign leaves the narrow field of view of the HuskyLens.

Communication between phone and Arduino

In the gold challenge, the group implemented a feature where the buggy could be controlled by phone via an app. To do so an app was created on MIT App Inventor however the communication is only one sided. Currently the phone sends signals constantly and the Arduino only connects if "Mario mode" is turned on. By making this communication two sided, the Arduino could send an initial signal to tell the phone to start sending data. This would provide for a more efficient and effective method of communication.

Tuning turning speeds

During our Gold demonstration, the buggy got "stuck" briefly before continuing its path along the track. This occurred due to the method in which we tuned our turning speeds. They were tuned with the batteries at full charge. As the group began to practice more and more, the batteries would deplete resulting in less power to the motors. As a result, the buggy was "stuck" briefly when turning the corner. To resolve this, the tuning could have been done with the batteries at 75% full charge or even 50% fully charge. This would reduce the chances of the buggy getting "stuck" when turning the corners.

Ethics:

Ethics are important when it comes to storing data, so keeping it in mind throughout the project is essential. In this project, there are some important factors with ethics, most of them surround the HuskyLens and how the data it processes is stored. The HuskyLens has many features, such as face recognition and tracking. It is vital that these features are never used with any malicious intent.

The HuskyLens works on machine learning, where it stores the data it has been trained on and hence can recognise it for future use. For this reason, it is the responsibility of the person teaching the HuskyLens to stay mindful about what exact data they are importing into the HuskyLens. If they are using facial recognition, they must have full consent of the person whose face is being recorded.

Another factor you need to take into account if you are training a machine learning algorithm is the dataset you are training the algorithm with. A biased dataset could result in a biased algorithm, which often times could be very harmful. This wasn't as big of an issue in this project as the dataset was small and based on QR codes, however it is very important to keep in mind for other similar projects with large datasets.

Part B – Hardware Design

Block Diagram:

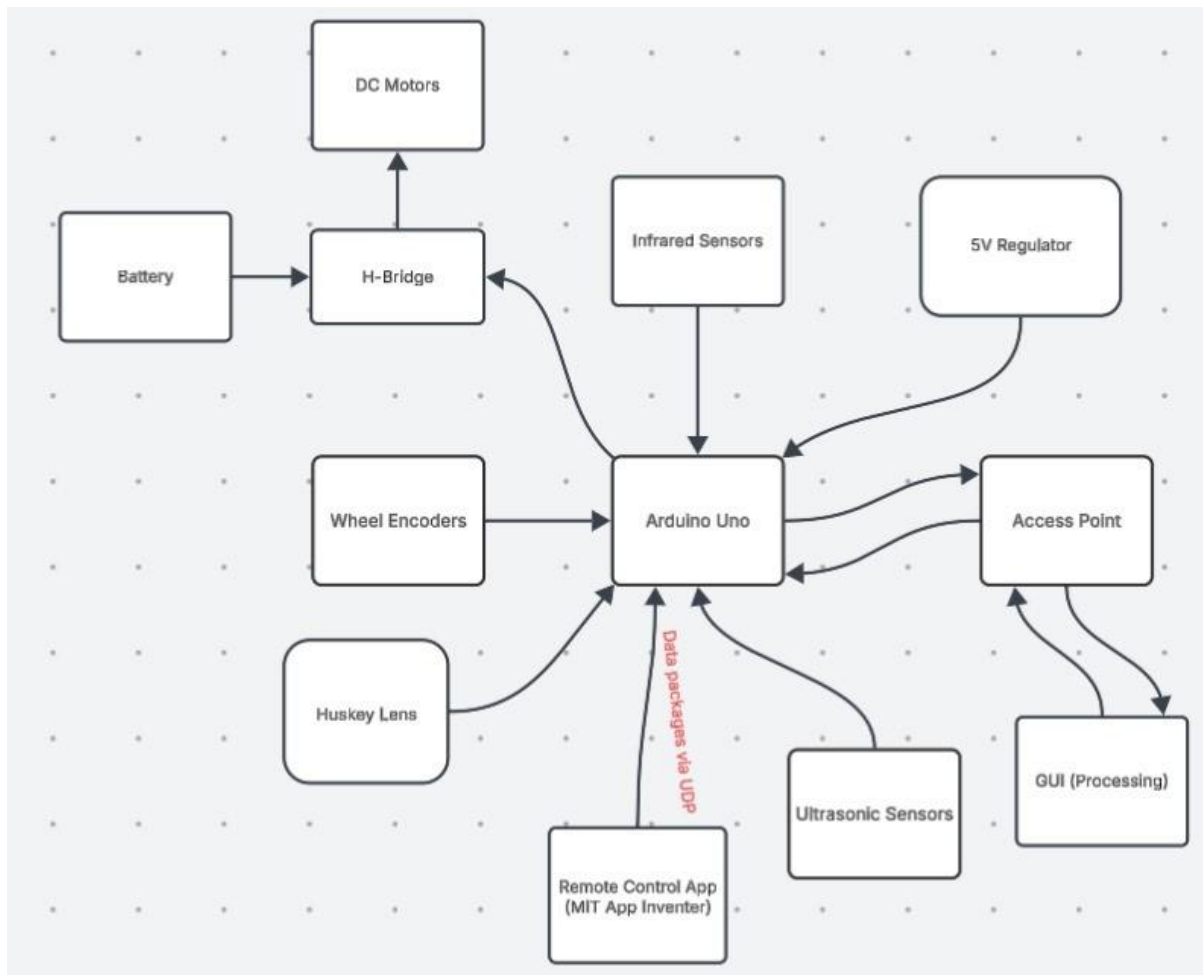


Figure 1: Updated Block Diagram for Silver and Gold

Circuit Diagram

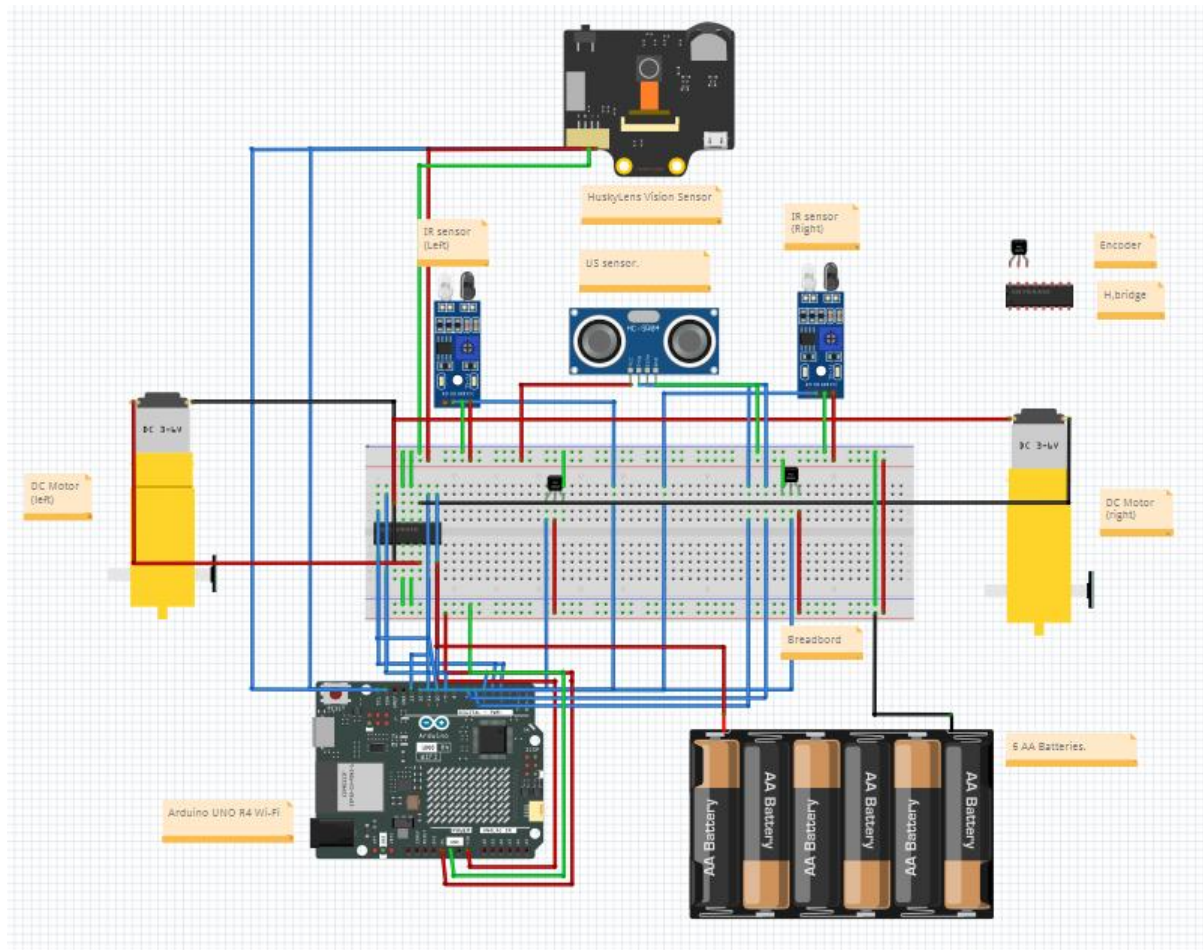


Figure 2: Updated Circuit Diagram for Silver

Analysis of the board

As with the silver challenge, the optimization of space on our board one of our most important objectives when designing the board. Even with the addition of the HuskyLens camera for the gold challenge, we kept our layout compact by carefully choosing which digital and analogue pins to use on the Arduino UNO. Each pin was selected to avoid wire crossing and to keep the wires as short as possible. This helped us keep the build tidy and reduced the chances of things coming loose. The layout still has room for improvement, but overall, the board worked well for our testing and goals in the gold challenge.

Challenges

The main challenges that we faced were...

Loose wires

This issue continued from the silver stage. With the added HuskyLens camera, it became even harder to spot when something came loose. A single wire disconnecting would affect the whole buggy and was hard to notice at first.

Different wheel power

Just like in the silver challenge, our buggy would still slightly drift to the right when driving straight. The left motor seemed stronger, which made it harder to follow the white line and take tight corners.

Low battery power

As we added the HuskyLens camera, it used more power, and we started noticing the motors didn't run as strongly when the batteries were low. Sometimes the buggy would take a moment to make a move.

Camera not staying upright

A new problem we had in the gold stage was the HuskyLens camera moving out of position. If it wasn't perfectly straight, it would miss the April-Tags or not read them properly. This made navigation less reliable.

The fixes to each problem and how to keep it from occurring again:

1. Loose wires
Make sure all wires are pushed in properly and do a quick check of the board before running any tests.
2. Wheel power difference
Adjust the motor power in the code so each wheel runs at a slightly different speed, or slightly angle the IR sensors to help it track better.
3. Low battery power
Charge the batteries regularly and avoid over-testing in one session so you have enough power for the camera and motors during important tests.
4. Camera position
Make sure the HuskyLens camera is fixed in place tightly and check it before every test, so it stays in the right position to detect tags.

Part C – Software Design

All parts of the software design section of the interim report were updated to describe the new solution using Domain and System Models, along with pseudo-code.

Domain Model

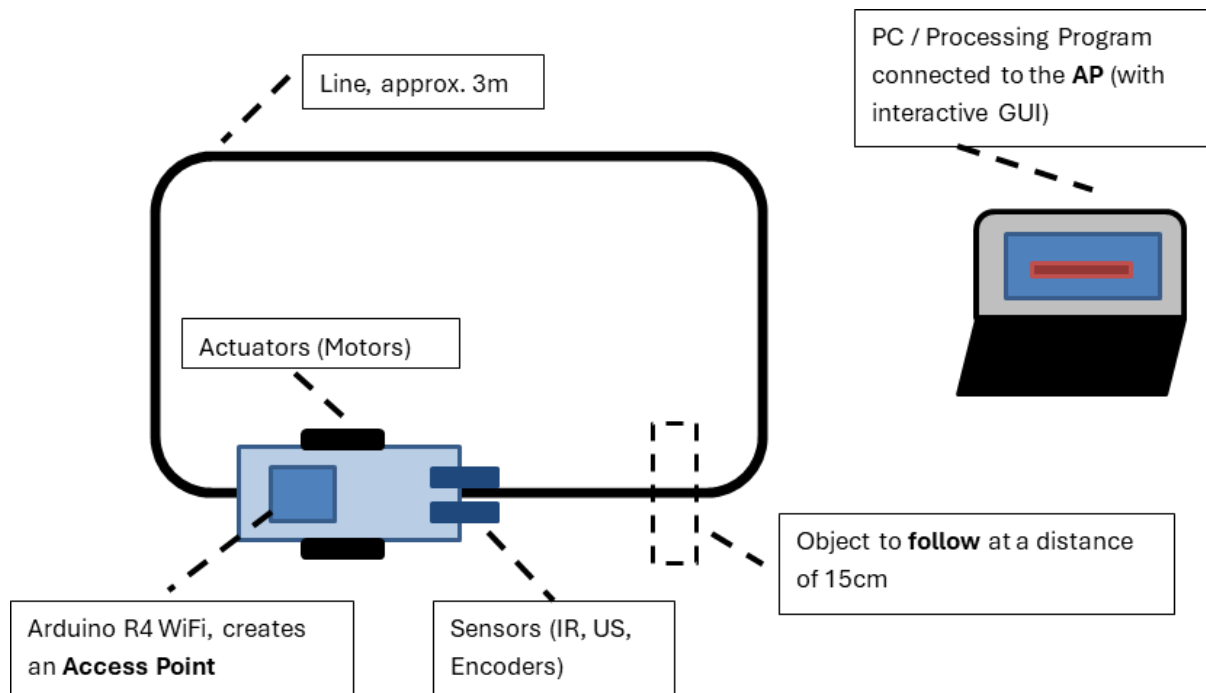


Figure 3: Updated Domain Model for Silver

For the Silver challenge, all rules explained in the previous report still apply, with some additional requirements:

The buggy's speed must be controlled via a **PID controller** with 2 different modes. The user must be able to enter a **goal speed** in cm/s into the GUI or select an **object following** mode such that the buggy will maintain a distance of 15cm to the closest detected object. Current mode, speed and distance to an object should all be displayed in the **updated GUI**.

The setup as shown in the Domain Model remains similar to bronze, except for a switch from communication via mobile hotspot to an **access point**. This decision was made due to issues arising when the phone used in bronze was replaced by a different model.

Since the silver challenge did not require any additional "elements" or hardware, most of the changes made can be seen in the System Model and pseudo-code.

System Model

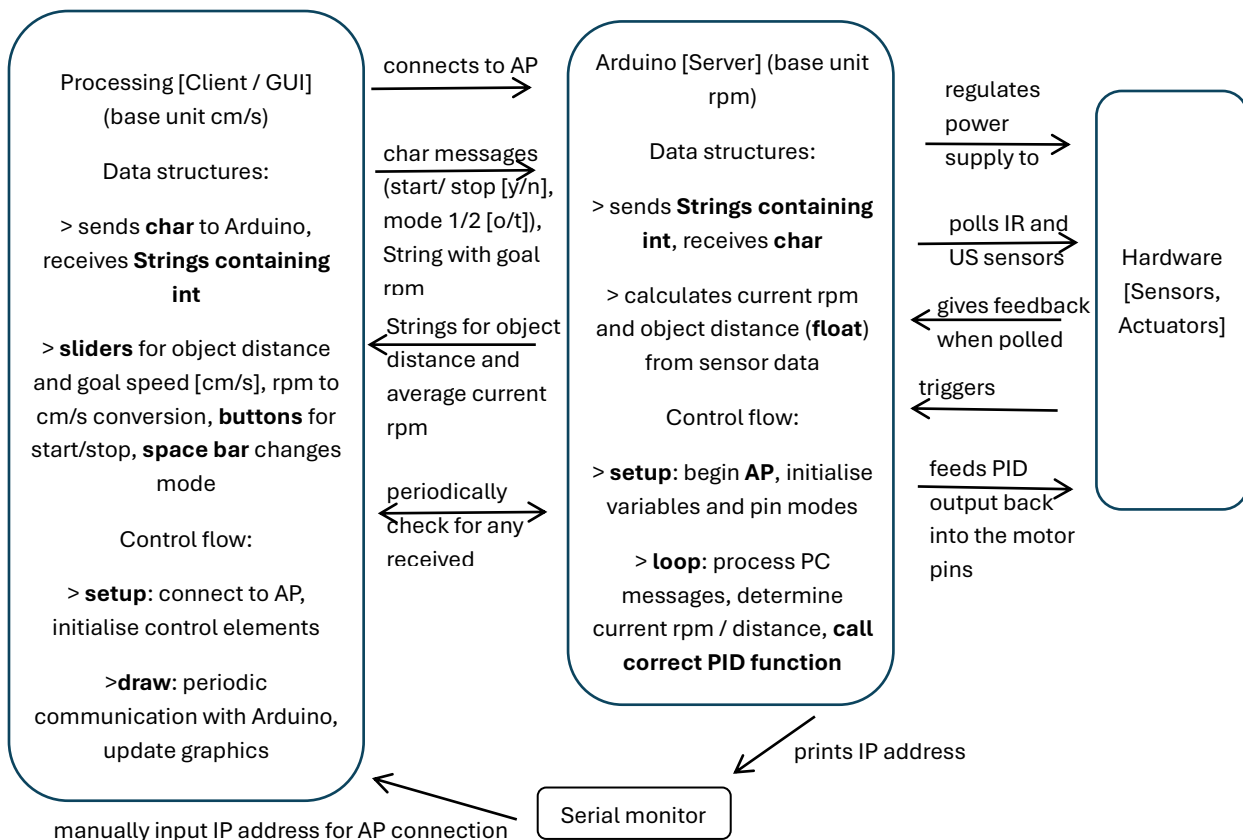


Figure 4: Updated System Model for Silver

The main changes in comparison to bronze are the implementation of two **PID** functions and an **increase** in the amount of **data** exchanged between server and client and displayed in Processing. The GUI needed to additionally display the current mode, actual speed, goal speed and object distance. The distance had already been implemented using a slider in bronze and the **goal speed** was added in a similar manner, using a slider from 0 to 20 cm/s to avoid excessively high velocities. For the **actual speed**, a meter was created with the help of the meter library and **mode changes** were triggered by the space bar and reflected on the GUI via a change in heading and colour scheme. The GUI for both modes can be seen in Appendix 0, Figure 11.

The method of **data transmission** remained the same in terms of sending characters and Strings containing integers and ensuring clear communication by sending an identifier and endline character with every distance and speed value. However, the PID functions had been tuned in the base unit RPM when it was announced that the display should use cm/s. Thus, an additional conversion step was added in processing to avoid impacting the PID constants through a change of unit.

Pseudo Code

The full code for silver is not separately show in the appendix because it is included in the gold files and all silver functionality was maintained.

Processing Pseudo Code

```
Import libraries "meter", "controlP5" and "net"
Declare global variables, client object, buttons, sliders, meter
Setup(){
Connect to access point
Send 'w' character to Arduino
    Initialise buttons, fonts and sliders, meter
Format meter
}
Draw(){
Set background to black
If mode == speed control {
Set button/slider/meter colours to blue
}
Else {
Set button/slider/meter colours to red
}

Sending to server -----
If loop counter % 10 == 0 { // every 10th loop
If start was last pressed {
Send 'y' character
}
Else {
Send 'n' character
}
If mode == speed control {
Send 'o' character
Get new goal speed from slider
Convert cm/s to rpm
Send new goal rpm
}
Else {
Send 't' character
}
}

    Increment loop counter

Receiving from server -----
If Arduino has sent something {
Read in identifier
switch (identifier) {
case 1:
Read in distance from object
Convert to integer
break
case 2:
Read in distance travelled
Convert to integer
break
```

```

case 3:
Read in current rpm
Convert to integer
Convert to cm/s
break
}
}

Update the meter // current speed in cm/s
Update the object distance slider
Update the encoder distance textbox

Draw white frames
    Delay by 10 ms
}

Interactive elements -----
Start button pressed() {
Set boolean "go" to true
}
Stop button pressed() {
Set boolean "go" to false
}
Space bar pressed() {
Change between modes // mode = !mode
}

```

Arduino Pseudo Code

The Arduino code for silver contains two header files which will be explained in pseudo code after the main ".ino" file. The second header file was only used for global variables and thus the explanation is kept short.

Main.ino

```

Include header files "WiFiS3.h" and "PID.h"
Setup() {
Declare pin modes
// motor&echo = output, IR&trig = input, encoder = input_pullup

Begin access point
Print IP address

Attach interrupts
Initialise motor pins // all set to LOW
Initialise speed to rpm_to_pwm(goal rpm)
Initialise time related variables
}

```

The rpm_to_pwm function will be discussed later in this section.

```

Loop() {
Receiving from client -----
If Processing has sent a message {
    Read in message
    If message is y {

```

```

Set boolean "go" to true
    }
    If message is n{
Set boolean "go" to false
    }
    If message is o {
Set mode to speed control
    Read in new goal rpm
    If new goal rpm is different from current {
        Update goal rpm
        Set speed to rpm_to_pwm(goal rpm)
    }
}
If message is t{
    Set mode to object following
}
}

```

```

Us Sensor -----
If loop counter % 10 == 0 // every 10th loop {
Set Us sensor trig pin HIGH for 10µs and then LOW
    Receive pulse with echo pin
Calculate distance from the pulse duration

Call obstacle_react(distance)
}

```

```

IR Sensors -----
If there is no obstacle closer than 15cm AND "go" is true{
Read in IR sensor values (HIGH/LOW)
If both see white {
Call stop()
}
Else if left sees white {
Call t_right() // right turn
}
Else if right sees white {
Call t_left() // left turn
}
Else {
Call moveForward()
}
}
Else {
Call stop()
}
}

```

```

PID control -----
If 200ms have passed since last call to PID {
If mode == speed control {
Add speedPID() to speed
}
If mode == object following {
Add distancePID() to speed
}
}
}

```

```

Sending to client -----
Convert object distance to a String
Send object distance String
Convert distance travelled to a String
Send distance travelled String
Convert current rpm to a String
Send current rpm String

}
Functions -----
ISR left() {
Increment left wheel counter
}
ISR right() {
Increment right wheel counter
}
Obstacle_react(distance) {
If distance is less than 15cm {
Set boolean "obstacle" to true
}
Else {
Set boolean "obstacle" to false
}
}
MoveForward() {
Set motor pins to forward
Set enable pins to "speed" pwm value
}
T_left() {
Set motor pins to forward
// high and low values are set for 4 intervals of the current
//speed
Set left enable to a low value
Set right enable to a high value
}
T_right() {
Set motor pins to forward
// high and low values are set for 4 intervals of the current
//speed
Set right enable to a low value
Set left enable to a high value
}
Stop() {
Set motor pins to LOW
}
Rpm_to_pwm(RPM) {
PWM = f(RPM)
Return PWM
}

```

A note on the rpm to_pwm() function: While testing the PID controller, a problem was observed when setting lower goal speeds. The buggy would stop and not be able to create enough torque to start moving again. This is why a function was implemented to initialize the speed, by estimating the PWM value needed through a calibration curve determined

experimentally. The Excel plot can be found in Appendix 0, Figure 9 and the line of best fit was found to be:

$$PWM \approx 6 * 10^{-6} * RPM^3 + 0.0016 * RPM^2 - 0.2853 * RPM + 60.977$$

PID.h

```
Include header file "variables.h"
SpeedPID() {
Compute time since last call to the function
Compute average wheel encoder counter
Compute current rpm

Update the array of the last 6 rpm values
Compute the average of the last 6 rpm values
```

The moving average of the RPM value is only used to send to Processing for a more realistic display of the current speed. The RPM used to compute the error for the PID calculation is only the current value and not an average since this has yielded better results in testing.

```
If buggy is currently turning{
Set error to a percentage of goal - current rpm
}
Else {
Set error to goal - current rpm
}

If the time elapsed is not 0{
Set rate error to change in error / time elapsed
}

If the buggy is actually moving {
Add error * elapsed time to error sum
}

If the error sum is getting very big {
Set error sum to 0
}
Set output to Kp * error + Ki * error sum + Kd * rate error
Update variables
Return output
}
```

The PID function for object following works the same except for the variable names and error calculation:

```
UsError = object distance - 15.0
```

Note how the order of current and goal value in the subtraction is reversed for the two functions. This is because the speed should be increased for mode 1 when the goal rpm is larger than the current one, whereas for mode 2 the buggy must get faster when the goal distance is smaller than the measured one.

Variables.h

Most of the exact variable names and initializations are not relevant to a pseudo code based explanation. Yet, the PID constants are worth mentioning:

```
Double Kp = 0.1, Ki = 0.00001, Kd = 0.001
```

```
Double UsKp = 1, UsKi = 0.00001, UsKd = 1
```

The integral constants are significantly smaller than the proportional and derivative components. This decision was made after considerable time spent tuning the constants and may be explained by the way the speed variable is updated using PID. Since less oscillation was achieved when adding the PID output to the current speed, rather than setting the speed equal to the output, a certain type of error integration is already happening. This eliminates the need for a significant Ki and may be the reason any attempts to increase this constant made the response of the system less desirable.

Part C Conclusion

To complete the silver challenge, the group opted for implementing two similar PID functions for the two modes **manually**, rather than using a library. This allowed for more control over certain checks and safety measures, like limiting the error in turns so the buggy does not overshoot on the straights. Another important feature is the **RPM to PWM** conversion function, which gives the PID controller a first estimate of what the motor power should be, making transitions between different goal speeds much smoother. The GUI was focused on a clearly visible transition between modes triggered by the **space bar** and the use of a **meter** to display the current speed.

Part D – Gold Challenge

The group attempted the gold challenge and completed the track successfully in 1 minute and 44 seconds (due to almost stopping in one corner). Additionally, the group presented a remote-control feature which allowed for the steering of the buggy to be determined by the orientation of a smart phone, similar to a video game steering wheel. This section will outline **only the additions** made since completing silver to avoid large repetitions from section C.

Domain Model

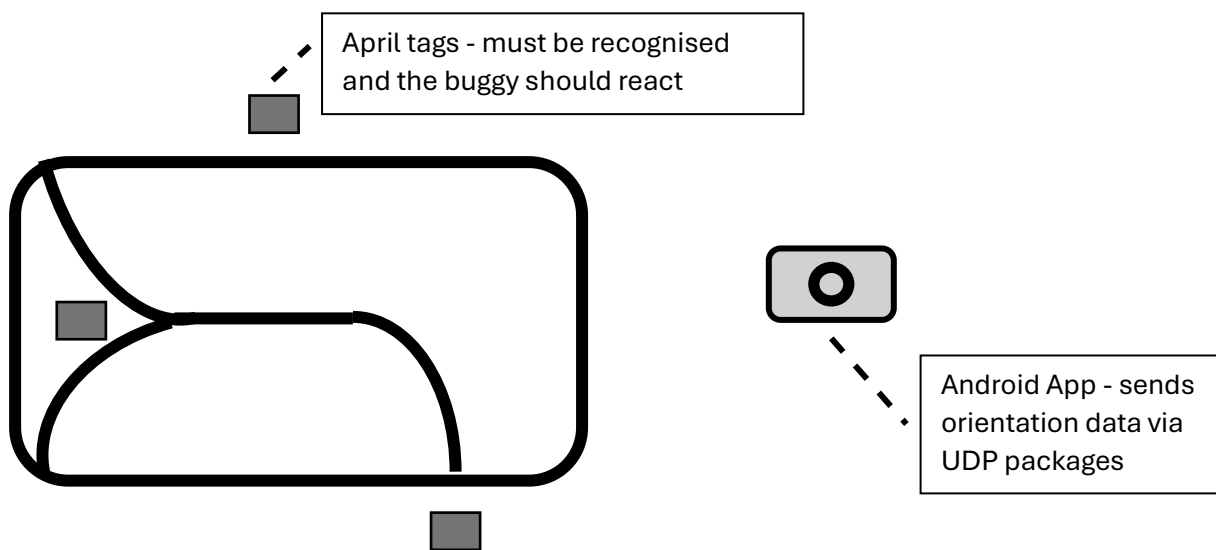


Figure 5: Additions to the Domain Model to achieve Gold

The only changes to the Domain model are the new track, April tag signs and the phone acting as a remote steering wheel. The rules of the gold challenge dictate that the buggy must use its HuskyLens camera to detect **four different tags** and respond by speeding up, slowing down, turning left or turning right, depending on the tag. In the GUI, it should also be visible when a tag is recognised and what that tag means.

System Model

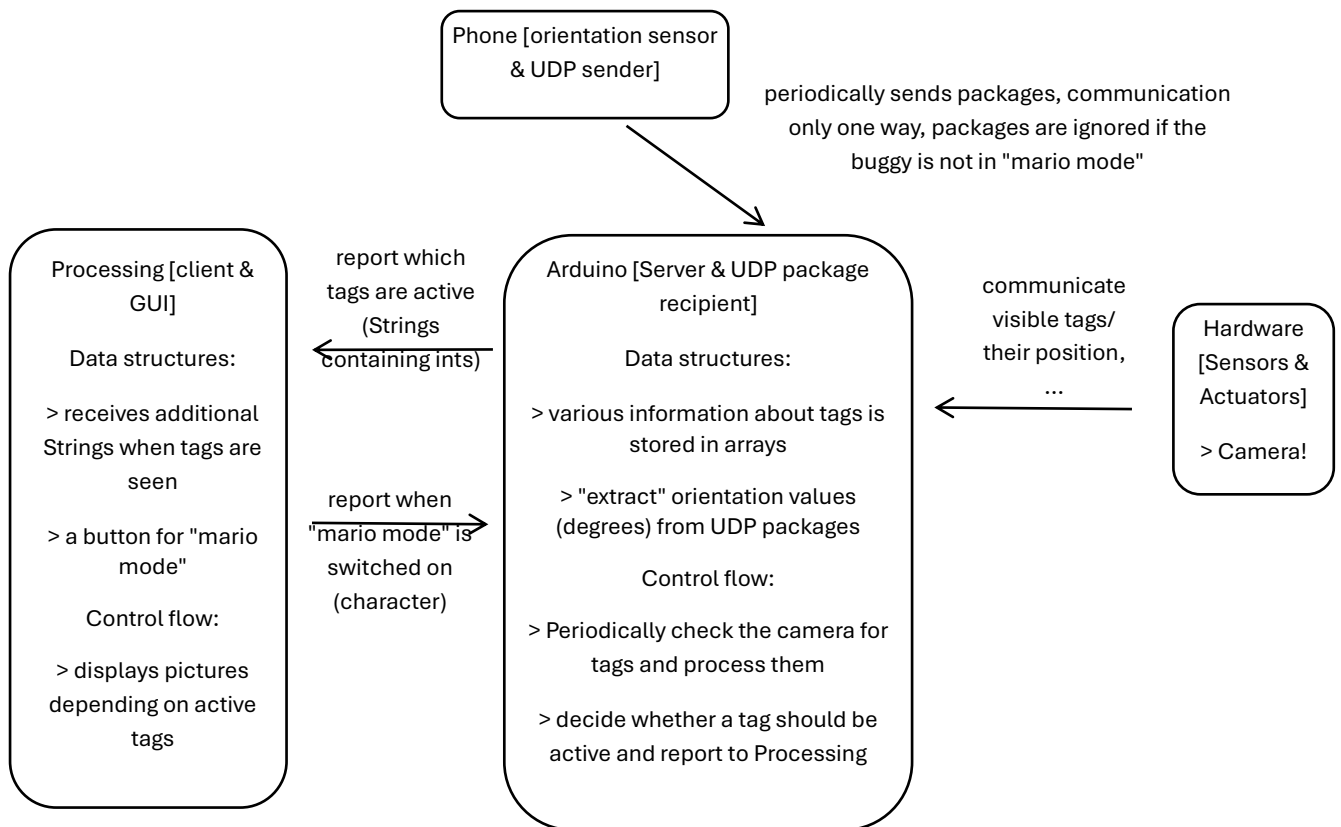


Figure 6: Additions to the System Model to achieve Gold

To meet the requirements of the gold challenge, the Arduino program had to be extended to process the camera input and make decisions about when to react to which tag. As will be shown in the pseudo code section, a lot of the data from the camera is dealt with using **arrays** to keep track of all the signs and quickly iterate over them with for loops. Communication with Processing was extended by assigning more integer **"identifiers"** to the tags and sending them as Strings. The GUI could then display pictures of the cars character Guido (after which the buggy is named) according to the recently seen tags.

The implementation of the remote-control feature relies on an Android app which sends **UDP (user datagram protocol)** packets containing the phone's orientation angle to the Arduino. While the buggy is not in remote-control mode, these messages are simply ignored, yet this was determined to be the best solution since it kept the connection to Processing intact. If the client-server model had been implemented using a mobile hotspot or external Wi-Fi, there may have been more options for responses back to the phone, but the access point was the only method not causing further issues.

Pseudo Code

Processing Pseudo Code

Processing_Gold.pde

Compared to silver, additional global variables, a “circle” class and images or audio files were introduced. The circle class will be shown separately like the previous header files.

```
Import library "sound"
Declare booleans for left, right, fast and slow
Declare images, audio
Declare six circle objects
...

Setup() {
Load images, audio
Initialize circle objects
...
}

Draw() {
If mario mode is active and music hasn't started {
Play rainbow road music
}
...
}
```

The section regarding data being sent to the Arduino had to be altered since Processing has to report when the remote-control mode is activated. Another small change was to only send the character for speed control mode ('o') when the slider for goal speed was adjusted. This is necessary because otherwise the buggy would have received a new target speed even when it is supposed to be going fast/ slow according to a tag. The option of intentionally overriding tags by moving the speed slider in processing was preserved this way.

```
Sending to server -----
If loop counter % 10 == 0 {
If NOT in remote-control mode{
If start was last pressed {
Send 'y' character
}
Else {
Send 'n' character
}
If mode == speed control AND slider changed {
Send 'o' character
Get new goal speed from slider
Convert cm/s to rpm
Send new goal rpm
}
Else {
Send 't' character
}
}
Else {
Send 'r' character
}
}
```

Increment loop counter

When receiving data from the server, tag identifiers were added to the switch case statement.

```
Receiving from server -----
If Arduino has sent something {
Read in identifier
switch (identifier) {
...
case 4:
Read in another identifier (int)
Set left and right true/ false based on int
case 8:
Read in another identifier
Set fast and slow true/ false based on int
}
}
```

After receiving information about the most recent tags, the corresponding pictures and animations needed to be displayed.

```
Displaying Tags -----
If left is true {
Clear previous direction images
Show left arrow
If slow is true {
Show slow-left image
}
Else if fast is true {
Show fast-left image
}
Else {
Show normal-left image
}
}
If right is true {
~repeat the same steps as for left~
}
If slow is true {
Play slow animation -> using circle.ascend()/display()
If no direction given {
Clear unnecessary images
Show slow image
}
}
    If fast is true {
~repeat the same steps as for slow~
}
}
If none are true {
Show normal image
Clear unnecessary images
}

Increment gui counter
If direction arrows are currently shown {
```

```

Move them every 10 counts
}

If remote-control is active {
Show mario image
}

Functions -----

Mario(){
Switch mario mode between true / false when clicked
}
Clear Image(dimensions){
Draw a black rectangle at those dimensions
}

```

Animations for fast and slow modes use the “circle” class.

Circle.pde

```

Class Circle {
Constructor with center coordinates and diameter
Ascend(){
Update x and y for animation
}
Display(){
Show an image with location and diameter of the circle
}
}

```

Arduino Pseudo Code

Gold.ino

Two additional header files were created for different driving modes and functions relating to the husky lens. For the UDP communication, a library was imported.

```

Include header files "SPI.h", "WiFiUdp.h", "camera.h", "driving.h"
...
Setup(){
Begin Matrix, Wire and UDP
Initialize tags to false, tag size/position/time arrays
    ...
}

```

The receiving section needed to account for remote-control mode.

```

Loop(){
Receive from client -----
If message is 'r' {
Set mode to 3 (mario mode)
}
}

```

If “mario mode” is activated, the Arduino needs to read in the UDP packets. Otherwise it should handle the camera input.

```

If mode is 3 (mario mode) {
Read in UDP packet
If a packet was received {

```

```

Convert packet to integer
Call remote_drive(packet value)
}
}
If mode is speed control {
Call get_tags()
}
If mode is speed control or object following {
Get IR sensor readings
If go is true and obstacle false {
Render LED display according to current tag
If left tag active {
Call drive_left()
}
Else if right tag active{
Call drive_right()
}
Else {
Call drive_normal()
}
}
}
}

```

The Strings sent to the client also needed to communicate the currently active tags.

```

Send to client -----
...
    Send 4 as a String (to signal left/right)
    Send identifier for which tag is true
// 5 = left, 6 = right, 7 = neither
    Send 8 as a String (to signal fast/slow)
    Send identifier for which tag is true
// 9 = fast, 10 = slow, 11 = neither

```

Camera.h

This header file contains the `get_tags` function and periodically handles the husky lens input. For each sign, the following information is recorded: Whether it is currently active, what size it was the last six times the camera was polled, the last position recorded for this tag and the last time instance where it was observed. This is more information than necessary, since tags were ultimately activated based on the size and time instance arrays alone, but this part of the code was kept for experimenting with different ways of making decisions about the signs.

```

Get_tags() {
If camera is not connected or nothing learned{
Print error statements
}
Else {
For each tag {
Move the values in the size array one spot over
}
While there is a tag to available {
Read in the tag
Update size array
Update position array
Update time instance array
}
}

```

```

For each tag {
Count how often it appeared in the last 6 frames
Record the size of its last appearance
If it was big enough AND appeared twice within a
    short time {
Set tag to true
If it is a direction tag {
Record the current distance
}
If it is a speed tag {
Update the goal speed for PID
}
}
}
If a direction tag has been active for more than 40 cm {
Set it back to false
}
If two direction tags are active at once {
Set the smaller one (last size) to false
}
If two speed tags are active at once {
Set the smaller one (last size) to false
}
}
}

```

Driving.h

The driving header file contains basic functionality like `moveForward()` or `t_right()` from silver, but the driving logic from the IR sensor section of the silver code was moved into a `drive_normal()` function. Additionally, the header file provides 3 other driving modes: driving left, right and remotely.

```

MoveForward() // as before
MoveForward_slow() // like MoveForward but with lower PWM values
MoveBackward() // as before
T_left() // as before
T_left_slow() // like T_left but with different PWM values
T_right() // as before
T_right_slow() // like T_right but with different PWM values
Stop() // as before

Drive_normal() // see IR sensor section of silver

```

The slow functions were intended to be used for slower turning speeds at junctions, yet it was experimentally determined that for left and right, the PWM values could not be lowered significantly. The slow forward movement on the other hand was found to be very useful when passing a junction with no tag for direction.

```

Drive_right(){
If left sees white and right sees black {
Call moveForward_slow()
}
Else if both see black {
Call t_left_slow()
}
Else if left sees black and right sees white {

```

```

Call t_right_slow()
}
Else {
Call t_right_slow()
}
}
Drive_left() {
If left sees black and right sees white {
Call moveForward_slow()
}
Else if both see black {
Call t_right_slow()
}
Else if left sees white and right sees black {
Call t_left_slow()
}
Else {
Call t_left_slow()
}
}
}

```

The idea behind these driving modes is to make the response to direction tags as robust as possible by changing the driving logic instead of turning for a certain time or similar approaches. The buggy follows the left or right edge of the line, e.g. for drive_right, it attempts to keep the left sensor on white and the right one on black.

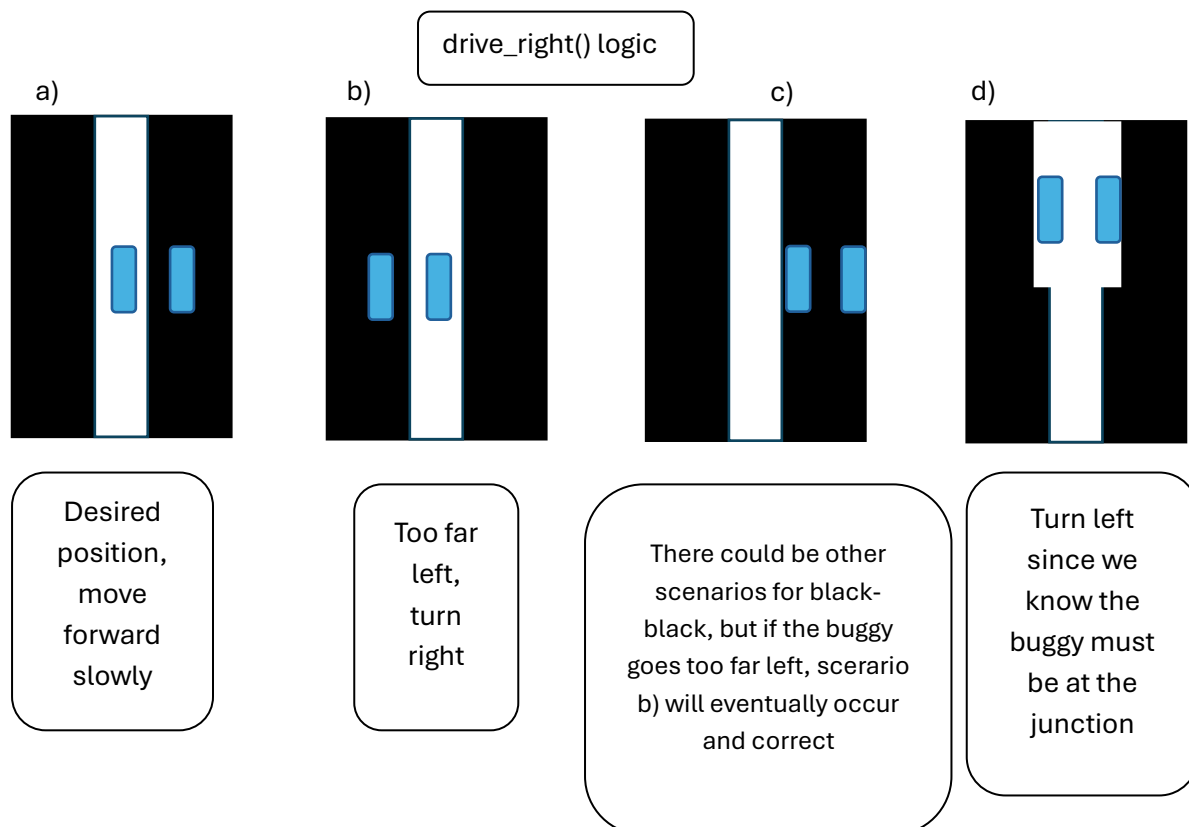


Figure 7: Driving Logic for Turning Right

The last driving mode is the one used for remote control. The phone sends orientation angles from -90° (tilted to the left) to 90° (tilted to the right).

```

Remote_drive(angle){
Map the angle onto pwm of the right motor (0 to 130)
Map the angle onto pwm of the left motor (0 to 95)
Set motor pins for forward drive
Set motor enable pins to corresponding speeds
}

```

The values are mapped as shown in Figure 6. This distribution is mainly due to the difference in performance between the left and right motors.

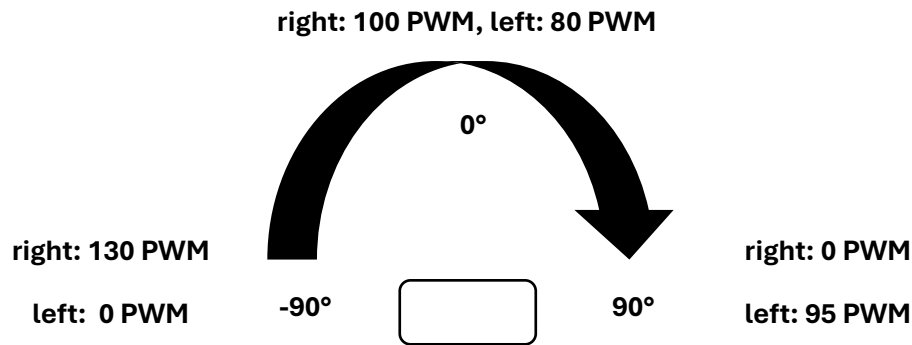


Figure 8: Phone Orientation to Buggy Speed Mapping

Android App

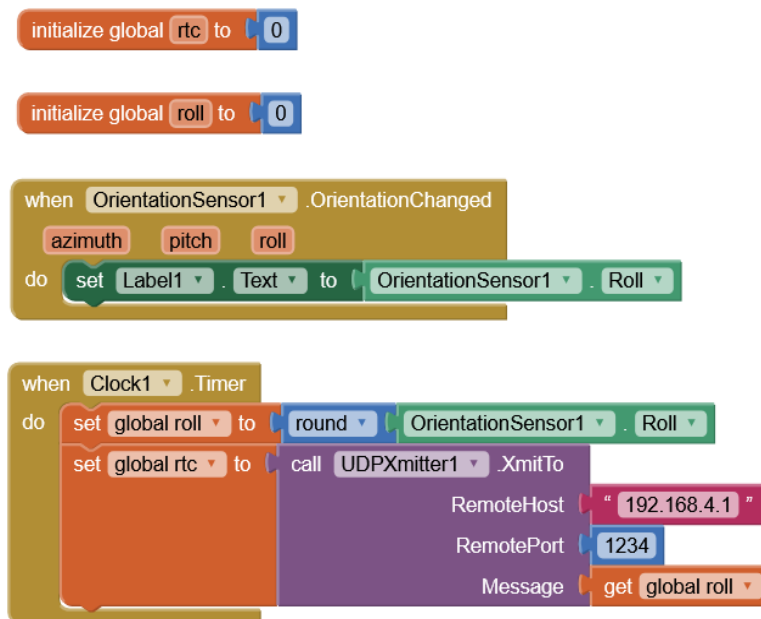


Figure 9: Android App Code using MIT App Inventor Blocks

The “App” was implemented using a website called “MIT App Inventor” and its code consists of different block elements shown in Figure 9. “Roll” refers to the parameter of the orientation sensor that is used to represent tilting, as shown in Figure 8. The UDP transmitter element was imported from a UDP library and requires the Arduino’s IP address (“Remote Host”) and the Port used for UDP communication (1234 was chosen here).

Part D Conclusion

The gold challenge was achieved through a combination of **data handling** in arrays and **WiFi communication** with set identifiers to avoid misinterpretation. Another key decision was the **driving logic** shown in Figure 6. For the remote control feature, the choice of **UDP** as the protocol was made to keep the connection to Processing intact, but this may be an area of improvement for the future.

Part E – Appendix

0. Additional Graphs and Images

The RPM to PWM conversion function used in the siler challenge is based of the graph of experimental results shown in Figure 10.

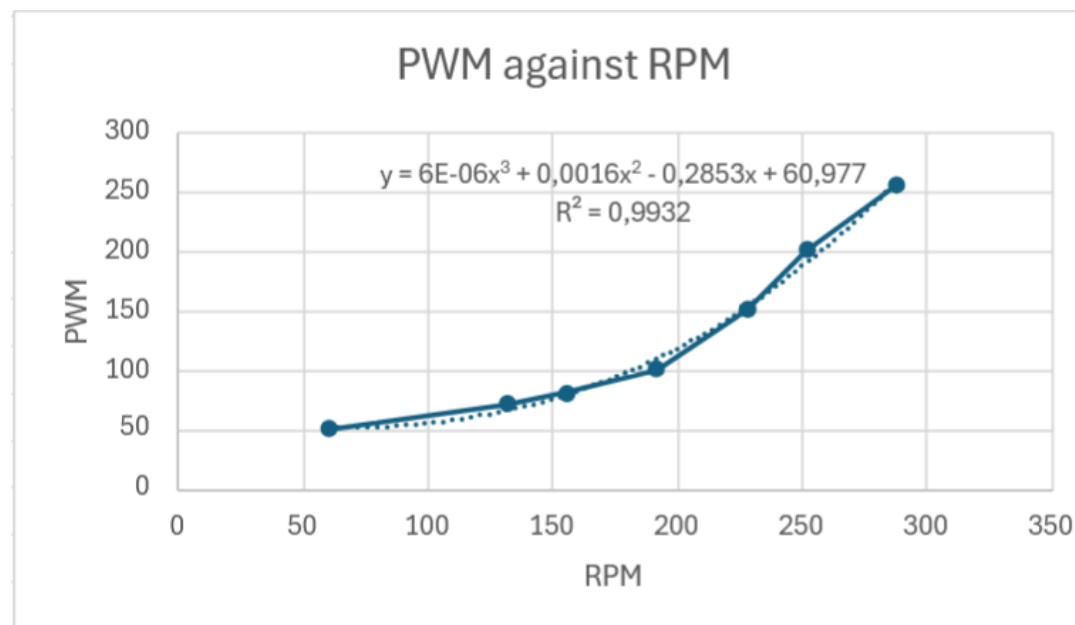


Figure 10: RPM to PWM Conversion Calibration Graph

Additionally, some screenshots of the GUI for silver and gold are included here:

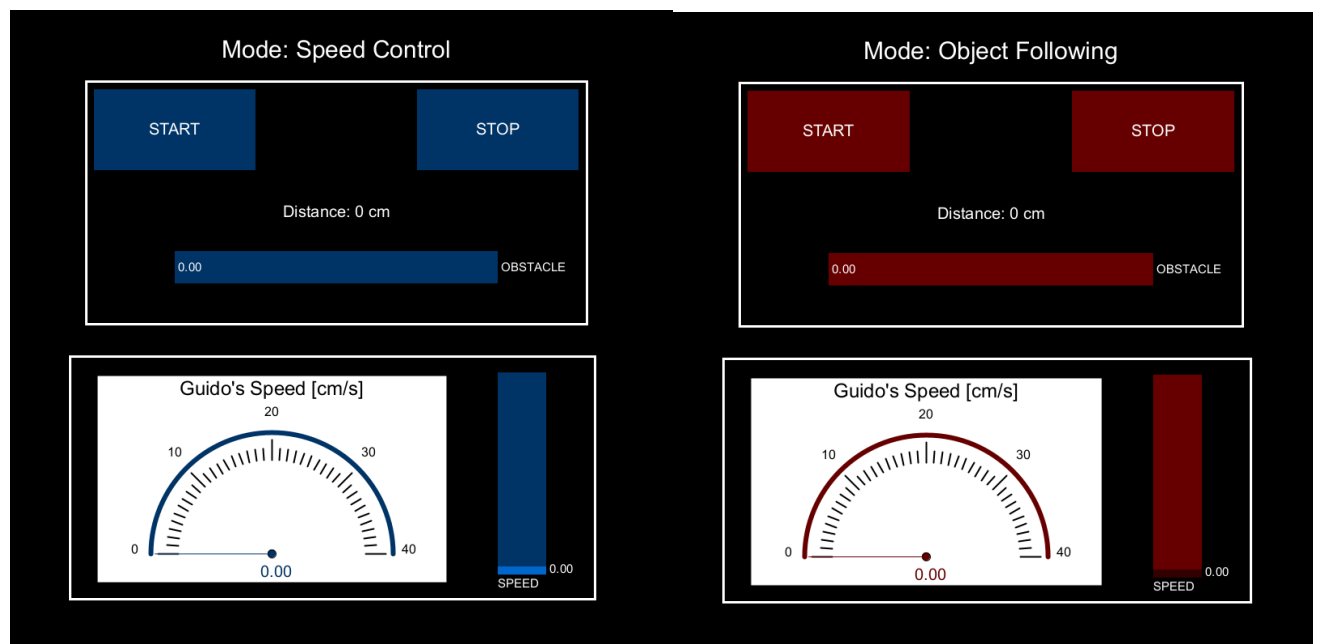


Figure 11: The Silver GUI for Speed Control (left) and Object Following (right)

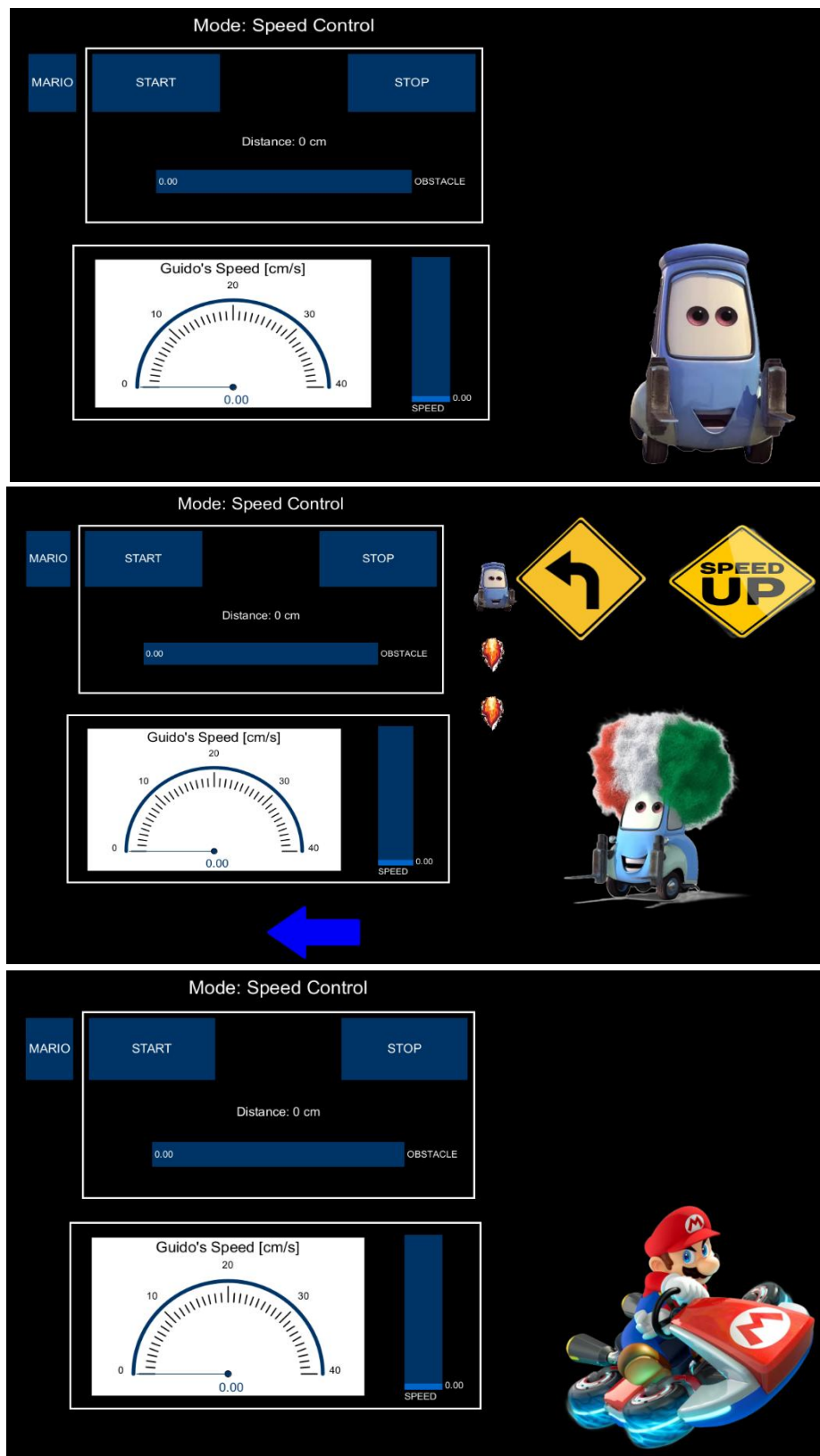


Figure 12: Examples of the Gold GUI ("normal", "left and fast", "Mario Mode")

Appendices I & II are submitted as separate files.

3. Version History

The change log includes changes made to achieve gold, since the Github repository was set up right after silver.

Note that changes are chronological starting at the bottom.

```
68ba29c - pbruedrich, 2025-04-11 : cleaning up the code
0eel3cf - farleynf, 2025-04-09 : final gold
1e3e146 - farleynf, 2025-04-09 : final processing version with mario
4dd2f2b - farleynf, 2025-04-09 : mariok
17a7677 - farleynf, 2025-04-09 : video upload
a11bf4e - pbruedrich, 2025-04-09 : small changes regarding remote
turning speeds
91f2b64 - pbruedrich, 2025-04-08 : Mario Kart Mode
5186ce6 - pbruedrich, 2025-04-08 : gold updates
62ddb4c - pbruedrich, 2025-04-07 : repeat the github setup because
of laptop issues
b7a7860 - jackdai21, 2025-04-07 : Add files via upload
57339b2 - jackdai21, 2025-04-06 : Add files via upload
dc951e3 - jackdai21, 2025-04-06 : Add files via upload
132c54c - jackdai21, 2025-04-06 : Update processing_gold.pde
8ddd4e2 - jackdai21, 2025-04-06 : Update Circle.pde
43b9330 - jackdai21, 2025-04-04 : Add files via upload
0fcd7ce - jackdai21, 2025-04-04 : Update processing_gold.pde
533773f - jackdai21, 2025-04-04 : Add files via upload
aa05c09 - jackdai21, 2025-04-03 : Update processing_gold.pde
f6978ce - jackdai21, 2025-04-03 : Update processing_gold.pde
53c94a9 - jackdai21, 2025-04-03 : Add files via upload
e4247d4 - pbruedrich, 2025-04-03 : receive tag info from arduino
5417b5a - pbruedrich, 2025-04-03 : sending data to processing about
which mode we're currently in
0facfa4 - pbruedrich, 2025-04-03 : pictures
5e6b166 - pbruedrich, 2025-04-03 : added gold processing
e959869 - pbruedrich, 2025-04-03 : debugging
1145be5 - pbruedrich, 2025-04-02 : wednesday session
fe89e8e - pbruedrich, 2025-04-01 : left and right turning
implementation
bb3fb36 - pbruedrich, 2025-03-31 : size adjustment for smaller sign
3cedca6 - pbruedrich, 2025-03-31 : driving header file, driving left
and right modes
80d27a5 - pbruedrich, 2025-03-31 : small change of size limitation
c1f3b29 - pbruedrich, 2025-03-31 : time test version
c63d343 - pbruedrich, 2025-03-31 : stop for tag 1
94286dc - pbruedrich, 2025-03-29 : add basic camera code to gold
file
e076768 - pbruedrich, 2025-03-29 : compensate for new motor
difference
f0da465 - pbruedrich, 2025-03-29 : header files
dd4e241 - pbruedrich, 2025-03-29 : just a rename
976bf2a - jackdai21, 2025-03-27 : Add files via upload
```